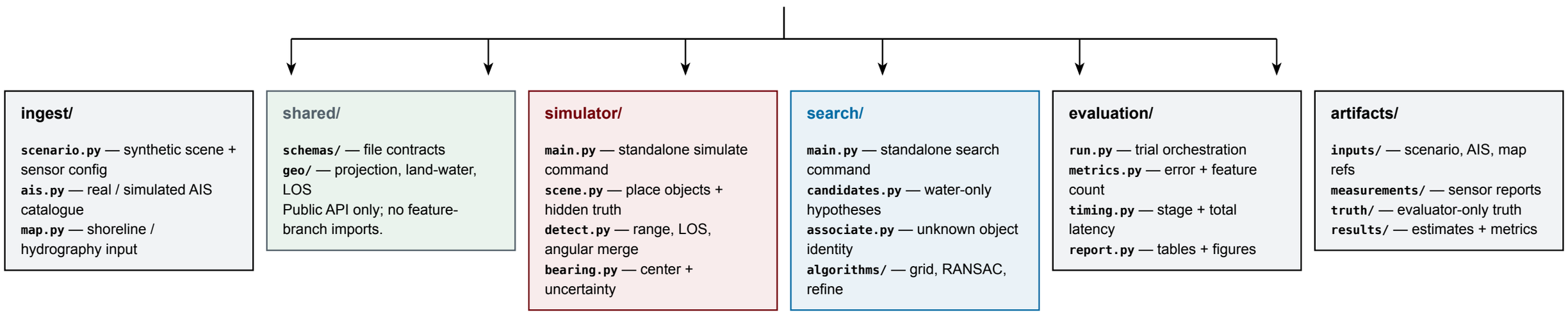
A rooted software package tree

One top-level entry point coordinates independent branches; sibling programs exchange files, not each other’s internal code.

main.py — package trunk / orchestrator

Parses the command, loads configuration, selects a run mode, starts the appropriate branch, and records run metadata. It passes versioned files and shared-service handles between branches; it does not contain sensor simulation or localization algorithms.



ingest/

`scenario.py` — synthetic scene + sensor config
`ais.py` — real / simulated AIS catalogue
`map.py` — shoreline / hydrography input

shared/

`schemas/` — file contracts
`geo/` — projection, land-water, LOS
Public API only; no feature-branch imports.

simulator/

`main.py` — standalone simulate command
`scene.py` — place objects + hidden truth
`detect.py` — range, LOS, angular merge
`bearing.py` — center + uncertainty

search/

`main.py` — standalone search command
`candidates.py` — water-only hypotheses
`associate.py` — unknown object identity
`algorithms/` — grid, RANSAC, refine

evaluation/

`run.py` — trial orchestration
`metrics.py` — error + feature count
`timing.py` — stage + total latency
`report.py` — tables + figures

artifacts/

`inputs/` — scenario, AIS, map refs
`measurements/` — sensor reports
`truth/` — evaluator-only truth
`results/` — estimates + metrics

Branch boundaries

The simulator branch never imports search internals, and search never imports simulator internals. Both may call the stable shared/geo API; they do not call one another. The simulator writes observations and hidden truth to separate artifacts. Main passes observations and AIS to search, then passes estimate plus truth to evaluation.

Expanded branch tree and file roles

Proposed directory and file names. Each branch has its own entry point and owns its implementation.

main.py → ingest/

`ingest/main.py` — entry point for ingestion commands
`ingest/scenario.py` — load configuration; generate object inventory with AIS-present flags
`ingest/ais.py` — read records; normalize time, position, type, dimensions, and AtoN type
`ingest/map.py` — load coastline / hydrography, CRS, and source metadata
Main chooses the ingest operation and writes normalized inputs for the later branch.

main.py → shared/

`shared/schemas/observation.py` — type label, global bearing, optional bearing sigma and apparent width
`shared/schemas/truth.py` — true boat pose and hidden scene IDs; evaluator only
`shared/schemas/estimate.py` — estimated pose, confidence, optional associations
`shared/geo/project.py` — convert lat/lon to local metric coordinates
`shared/geo/land_water.py` — build and query versioned land-water mask
`shared/geo/line_of_sight.py` — ray / segment test; same API for both programs

main.py → simulator/

`simulator/main.py` — standalone simulation command
`simulator/scene.py` — place boat and objects; preserve hidden truth
`simulator/visibility.py` — apply range gate, then shared land-occlusion query
`simulator/detection.py` — merge angularly close detections; select reported type
`simulator/bearing.py` — center of object's angular extent; add noise / uncertainty
`simulator/write.py` — write measurements, truth, and scenario AIS catalogue separately

main.py → search/

`search/main.py` — standalone localization command
`search/inputs.py` — read measurements + AIS; reject truth input
`search/candidates.py` — generate coarse candidates on water
`search/association.py` — compare reported type / bearing to unknown AIS identity
`search/scoring.py` — bearing residual, uncertainty weight, LOS consistency
`search/algorithms/grid.py` — exhaustive or hierarchical grid
`search/algorithms/ransac.py` — sample correspondence hypotheses
`search/algorithms/refine.py` — refine best candidate positions
`search/write.py` — write estimate + diagnostics

main.py → evaluation/

`evaluation/main.py` — standalone evaluation / benchmark command
`evaluation/run.py` — run fixed cases across search methods
`evaluation/metrics.py` — position error, success, minimum features
`evaluation/timing.py` — per-stage, end-to-end, CPU / GPU timing
`evaluation/report.py` — write CSV / JSON and publication figures

main.py → artifacts/

`inputs/scene.json` — simulator scene configuration
`inputs/ais.json` — AIS catalogue passed to search
`inputs/map/` — versioned map/mask data
`runs//measurements.json` — search input
`runs//truth.json` — evaluation input only
`runs//estimate.json` — search output
`runs//metrics.json` — evaluation output

Execution path

`main.py → ingest → simulator → measurements.json → search → estimate.json → evaluation`
`simulator → truth.json → evaluation only`
The map and AIS inputs are passed as data; search does not reach into simulator to get them.